

Esami API Programming 2021 in Rust

15 febbraio 2021

La struttura generica **Exchanger<T>** permette a due thread di scambiarsi un valore di tipo **T**. Essa offre esclusivamente il metodo pubblico **fn exchange(&self, t: T) -> Option<T>**, che blocca il thread chiamante senza consumare CPU fino a che un altro thread non invoca lo stesso metodo sulla stessa istanza. Quando questo avviene, il metodo restituisce l'oggetto passato come parametro dal thread opposto.

Si implementi tale struttura, usando la libreria standard di Rust.

19 giugno 2021

Si implementi una struttura **SingleThreadExecutor<F: FnOnce() + Send + 'static>** che realizzi il concetto di **ThreadPool** basato su un singolo thread.

- **Coda dei compiti:** La struttura deve utilizzare una coda per accodare i compiti da eseguire. I compiti vengono rappresentati come funzioni o chiusure che soddisfano i tratti **FnOnce() + Send**.
- **Metodo submit(...):** Attraverso questo metodo, è possibile affidare un compito all'istanza di **SingleThreadExecutor**. I compiti vengono accodati e saranno disponibili per l'elaborazione. Se l'esecutore è stato chiuso, eventuali tentativi di invocare **submit(...)** devono restituire un errore.
- **Metodo close():** Questo metodo deve impedire l'ulteriore accodamento di compiti. Dopo la chiamata a **close()**, eventuali invocazioni di **submit(...)** devono fallire con un errore. Tuttavia, i compiti già accodati devono poter essere eseguiti.

Si implementi tale classe usando le funzionalità offerte dalla libreria standard di **Rust**, definendo tutte le parti eventualmente mancanti nella definizione della classe. Si faccia attenzione al fatto che il codice che può essere sottomesso all'esecutore è arbitrario e può contenere richieste di sottomissione di ulteriori compiti allo stesso esecutore.

```
struct SingleThreadExecutor<F: FnOnce() + Send> {...}

impl <F: FnOnce() + Send> SingleThreadExecutor<F> {
    pub fn new() -> (Self, Receiver<Box<F>>);
    pub fn submit(&self, task: F) -> Result<(), String>;
    pub fn close(&mut self);
}
```

5 luglio 2021

Si implementi in linguaggio Rust una struttura generica `Buffer<T>` che modelli una struttura dati condivisa tra due thread concorrenti, uno produttore e uno consumatore. La struttura deve consentire al produttore di inserire valori e notificare la terminazione della produzione, mentre il consumatore può richiedere valori in modalità FIFO, rispettando le seguenti regole:

- **Metodo `next(...)`:** Il thread produttore utilizza questo metodo per aggiungere un nuovo valore al buffer. Se è stata invocata una chiamata a `terminate()` o `fail(...)`, il metodo deve fallire lanciando un'eccezione e il buffer deve rimanere inalterato.
- **Metodo `terminate()`:** Il thread produttore utilizza questo metodo per notificare che non saranno disponibili ulteriori valori. Dopo questa chiamata, il buffer non accetta più nuovi valori. Se non ci sono valori nel buffer, il metodo `consume()` deve restituire `None`.
- **Metodo `fail(...)`:** Il thread produttore utilizza questo metodo per notificare un errore e indicare che non saranno disponibili ulteriori valori. Dopo questa chiamata, il buffer non accetta più nuovi valori. Se non ci sono valori nel buffer, il metodo `consume()` deve restituire un errore.
- **Metodo `consume()`:** Il thread consumatore utilizza questo metodo per prelevare un valore dal buffer in modalità FIFO. Se non ci sono valori disponibili, il metodo si blocca in attesa di nuovi valori o di una condizione di terminazione, senza consumare cicli di CPU. Se è stato invocato `terminate()` e non ci sono valori, restituisce `None`. Se è stato invocato `fail(...)` e non ci sono valori, rilancia l'eccezione specificata.

```
impl <T: Send> Buffer<T> {  
    pub fn new() -> Self;  
  
    // Metodi relativi alla produzione di valori  
    pub fn next(&self, value: T);  
    pub fn terminate(&self);  
    pub fn fail(&self, error: Box<dyn Any + Send>);  
  
    // Metodo relativo al consumo di valori  
    pub fn consume(&self) -> Result<Option<T>, Box<dyn Any + Send>>;  
}
```

2 settembre 2021

In un sistema concorrente sono presenti due gruppi di thread, detti rispettivamente produttori e consumatori, che si appoggiano ad una struttura dati condivisa per passarsi dati generici di tipo `T`. La struttura dati è *thread-safe* ed implementa il concetto di buffer circolare: al suo interno ospita un array di `N` elementi (con `N` specificato a livello di tipo) nel quale vengono depositati i valori ricevuti dai singoli produttori, in attesa che vengano passati ad un consumatore qualunque che ne faccia richiesta. I dati sono trattati in modalità FIFO (First-In-First-Out).

- **`pub fn insert(&self, t: T) {...}`** → Inserisce un elemento. Se il buffer risulta pieno nel momento in cui un produttore prova ad inserire un nuovo valore, l'operazione si blocca, senza consumare CPU, in attesa che si crei uno spazio a seguito della conclusione di un'operazione di lettura da parte di un consumatore.
- **`pub fn extract(&self) -> T {...}`** → Estrae un elemento. Analogamente, se un consumatore prova a leggere un valore mentre il buffer è vuoto, deve attendere, senza consumare CPU, che un produttore inserisca un nuovo valore.

Per generalità, accanto alle operazioni base di inserimento ed estrazione di un valore, la struttura dati offre anche una coppia di operazioni con attesa limitata temporalmente.

- **`pub fn try_insert_for(&self, t: T, d: Duration) -> TimeoutResult {...}`** → Cerca di inserire un elemento. Restituisce se l'attesa è scaduta o meno.
- **`pub fn try_extract_for(&self, d: Duration) -> (Option<T>, TimeoutResult) {...}`** → Cerca di estrarre un elemento, e restituisce una tupla con il possibile valore estratto e il risultato dell'attesa.

Si implementi la classe generica di seguito proposta utilizzando le funzionalità offerte dal linguaggio **Rust**, aggiungendo le parti eventualmente necessarie per ottenere il comportamento descritto.

```
struct CircularBuffer<T: Send + 'static> {...}

impl<T: Send + 'static> CircularBuffer<T> {...}
```

18 ottobre 2021

La struttura generica **Processor<T>** consente a un insieme di thread produttori di inviare oggetti istanza del tipo T (che si assume copiabile) a un thread consumatore, il cui comportamento è definito tramite una funzione fornita come parametro del costruttore.

Il costruttore di tale struttura riceve, come parametro, una funzione (o una closure) che accetta un argomento di tipo T e restituisce (). La struttura fornisce una coda per gestire gli oggetti inviati dai produttori e offre i seguenti metodi:

1. **Metodo send(&self, item: T):** Permette ai produttori di sottomettere un oggetto da elaborare. L'oggetto viene inserito in una coda in attesa che il thread consumatore esterno lo elabori. Se il metodo close(...) è stato invocato, eventuali chiamate a send(...) devono generare un errore o produrre un comportamento indefinito, a scelta dell'implementazione.
2. **Metodo close(&self):** Segnala la fine dell'accettazione di nuovi elementi. Dopo l'invocazione di questo metodo:
 - Non sarà più possibile inviare nuovi dati tramite send(...).
 - Il metodo non ritorna fino a quando la coda non è vuota e tutte le operazioni di elaborazione sono state completate.

La struttura **Processor<T>** deve garantire la sincronizzazione tra i produttori e il consumatore esterno utilizzando primitive di sincronizzazione di Rust. La logica di elaborazione e il ciclo di vita del thread consumatore devono essere gestiti esternamente.

Viene fornita la seguente dichiarazione di struct Rust come punto di partenza:

```
impl<T: Send> Processor<T> {  
    fn new<F>(f: F) -> Self where F: Fn(T) + Send {}  
    fn send(&self, item: T) {}  
    fn close(&self) {}  
}
```